BookLeaf Publishing

Backend Deep-Dive & Code Architecture Guide

Complete Line-by-Line Breakdown, Production Code Snippets, and Technical Interview Q&A

1. Architectural Decisions & Technology Trade-offs

BookLeaf Publishing handles 1,200+ titles monthly with over 22,000 catalog entries. The backend architecture was engineered to balance high concurrent I/O throughput, rock-solid author data privacy, and minimal AI inference cost.

Why We Chose Each Component (Trade-off Analysis)

• **Node.js + Express 5 vs. Python (FastAPI/Django):** Express runs on V8's asynchronous, non-blocking event loop. Support ticketing spend 90% of execution time waiting on network I/O (MongoDB queries, external LLM API calls). Node.js handles thousands of concurrent I/O requests per second with negligible memory footprint, whereas multi-threaded Python WSGI servers consume substantially more RAM per thread.

• **MongoDB (Mongoose 9) vs. PostgreSQL (Relational SQL):** Publishing domain models are semi-structured: books in production have null publication dates and MRPs, while live books have sales arrays. Furthermore, support tickets require embedded message histories (responses) and private administrative logs (internalNotes). In PostgreSQL, this requires 3 normalized tables and multi-table JOINs on every query. MongoDB stores them as subdocuments in a single BSON document, enabling single-seek reads and atomic array pushes ($push).

• **HTTP-Only JWT Cookies vs. LocalStorage Authorization Header:** Storing JWTs in localStorage exposes the application to Cross-Site Scripting (XSS) attacks; any injected client script can read localStorage.setItem() tokens. Storing JWTs inside HTTP-only cookies with 'sameSite: lax' ensures the browser automatically manages token transmission, while client-side JavaScript is strictly blocked from reading or exporting the session token.

• **OpenRouter (gpt-4o-mini) vs. Claude 3.5 Sonnet / GPT-4o:** OpenAI's gpt-4o-mini costs ~$0.15 per million input tokens (~95% cheaper than GPT-4o or Claude 3.5 Sonnet) while providing ~600-800ms response times. For deterministic categorization and policy-constrained support replies, smaller instruction-tuned models deliver equivalent accuracy at a fraction of the operational cost.

• **Non-Blocking Fire-and-Forget AI vs. Synchronous Awaiting:** When an author creates a ticket, the server writes to MongoDB and immediately returns HTTP 201. The AI classification call runs asynchronously in the background. If the LLM provider experiences network latency or rate-limiting, the author's submission is never delayed.

2. Production Code Walkthrough by File

A. Application Entry Point: server/server.js

server.js configures the middleware pipeline, sets security boundaries, and guarantees that the database connects before the HTTP listener opens.

// server/server.js — Middleware & Database Initialization

```
// Rate limiting: 100 requests per 15-minute window per IP
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: { success: false, message: "Too many requests. Please try again later." }
});
app.use(limiter);

// CORS: explicitly binds to frontend port and enables cookie transmission
app.use(cors({
  origin: process.env.CLIENT_URL || "http://localhost:5173",
  credentials: true
}));

app.use(express.json());
app.use(cookieParser());

// Server Startup: waits for DB connection BEFORE opening port
async function startServer() {
  await connectDB();
  app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
}
```

• **Why rate limiting is critical:** Guards login endpoints against automated credential stuffing and prevents malicious actors from spamming ticket creation to exhaust OpenRouter AI API credits.

• **Why connectDB is awaited:** Starting Express before the database connection establishes causes initial user requests to throw connection pool timeout errors. Awaiting connectDB() guarantees that every handled request has an active database connection.

B. Resilient Database Layer: server/config/db.js

Mongoose connection logic configured with an explicit timeout to prevent silent hanging processes during cloud or network outages.

// server/config/db.js — Fail-Fast Connection Setup

```
const mongoose = require("mongoose");

async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 10000, // 10-second fail-fast timeout
    });
    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    throw error;
  }
}
```

• **Technical purpose of serverSelectionTimeoutMS:** By default, Mongoose may wait 30 seconds before timing out when an IP is blocked (e.g. MongoDB Atlas whitelist restriction). Setting a 10s timeout allows system managers and nodemon to detect connectivity problems quickly.

C. Data Models & Schemas: server/models/

// server/models/users.js — Password Isolation at Schema Level

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: {
    type: String,
    required: true,
    select: false // NEVER returned in normal queries
  },
  role: { type: String, enum: ["author", "admin"], default: "author" },
  authorId: { type: String, default: null } // e.g. "AUTH001"
}, { timestamps: true });
```

• **Why select: false is a critical security pattern:** Without 'select: false', running User.find() or User.findById() returns the bcrypt password hash. If an endpoint serializes the user object into JSON, the hash is exposed to the client. With 'select: false', Mongoose excludes the password field unless explicitly requested via .select('+password') in loginController.

// server/models/tickets.js — Multi-Array Communication Schema

```
const ticketSchema = new mongoose.Schema({
  authorId: { type: String, required: true, index: true },
  bookId: { type: String, default: null }, // null = "General / Account Level"
  subject: { type: String, required: true, trim: true },
  description: { type: String, required: true, trim: true },
  category: {
    type: String,
    enum: ["Royalty & Payments", "ISBN & Metadata Issues", "Printing & Quality",
           "Distribution & Availability", "Book Status & Production Updates", "General Inquiry"],
    default: "General Inquiry"
  },
  priority: { type: String, enum: ["Critical", "High", "Medium", "Low"], default: "Medium" },
  status: { type: String, enum: ["Open", "In Progress", "Resolved", "Closed"], default: "Open" },
  assignedTo: { type: mongoose.Schema.Types.ObjectId, ref: "user", default: null },
  aiClassified: { type: Boolean, default: false },
  aiDraft: { type: String, default: "" }, // Cached draft
  responses: [{ message: String, createdAt: { type: Date, default: Date.now } }],     // Author visible
  internalNotes: [{ note: String, createdAt: { type: Date, default: Date.now } }]    // Admin ONLY
}, { timestamps: true });
```

• **Subdocument segregation:** Separating 'responses' and 'internalNotes' allows the application to cleanly project author-safe data while keeping internal operational discourse strictly confidential.

D. Authentication & Role Middlewares: server/middlewares/

// server/middlewares/auth.js & adminsOnly.js — RBAC Enforcement

```
// auth.js: Validates session cookie and decorates request
async function authFn(req, res, next) {
  const userCookie = req.cookies?.bookleaf;
  if (!userCookie) return res.status(401).json({ success: false, message: "unauthorized" });

  try {
    const decoded = jwt.verify(userCookie, process.env.SECRET_KEY);
    const user = await User.findById(decoded.id);
    if (!user) return res.status(401).json({ success: false, message: "user not found" });

    req.userId = decoded.id;
    req.user = user; // Decorates req with role and authorId
    next();
  } catch {
    return res.status(401).json({ success: false, message: "invalid token" });
  }
}

// adminsOnly.js: Role guard
function adminOnly(req, res, next) {
  if (req.user.role !== "admin") {
    return res.status(403).json({ success: false, message: "admins only" });
  }
  next();
}
```

• **Stateless verification:** The JWT is decoded, validated against the database, and injected into the request object. Controllers downstream can immediately inspect req.user.role and req.user.authorId without re-querying authentication.

E. Asynchronous Ticket Controller: server/controllers/ticketController.js

// ticketController.js — Non-Blocking Async Ticket Creation

```
async function createTicket(req, res) {
  const { bookId, subject, description } = req.body;
  if (!subject || !description) {
    return res.status(400).json({ success: false, message: "subject and description required" });
  }

  // 1. Create ticket instantly with safe defaults
  const ticket = await Ticket.create({
    authorId: req.user.authorId,
    bookId: bookId || null,
    subject,
    description
  });

  // 2. Fire AI classification in background (NON-BLOCKING)
  classifyTicket(subject, description)
    .then((result) => {
      return Ticket.findByIdAndUpdate(ticket._id, {
        category: result.category,
        priority: result.priority,
        aiClassified: true
      });
    })
    .catch((err) => console.log("AI classify failed:", err.message));

  // 3. Return response immediately (sub-100ms response time!)
  return res.status(201).json({ success: true, message: "ticket created", ticket });
}
```

// ticketController.js — Priority Queue Sorting Algorithm

// ticketController.js — Priority Queue Sorting Algorithm

```
// Numerical ranking for business urgency
const PRIORITY_RANK = { Critical: 0, High: 1, Medium: 2, Low: 3 };

async function getAllTickets(req, res) {
  const { status, category, priority } = req.query;
  const filter = {};
  if (status) filter.status = status;
  if (category) filter.category = category;
  if (priority) filter.priority = priority;

  const tickets = await Ticket.find(filter);

  // Custom multi-attribute sort: Urgent first, then Oldest first
  tickets.sort((a, b) => {
    return (
      PRIORITY_RANK[a.priority] - PRIORITY_RANK[b.priority] ||
      a.createdAt - b.createdAt
    );
  });

  return res.status(200).json({ success: true, tickets });
}
```

• **Why in-memory comparator was chosen:** MongoDB standard string sorting orders alphabetically ('Critical', 'High', 'Low', 'Medium'), which places 'Low' before 'Medium'. Mapping priorities to numerical ranks ensures that Critical (0) and High (1) always float to the top of the operations queue.

// ticketController.js — Data Projection & Privacy Sanitization

```
const AUTHOR_HIDDEN_FIELDS = "-internalNotes -aiDraft";

async function getMyTickets(req, res) {
  const tickets = await Ticket.find({ authorId: req.user.authorId })
    .select(AUTHOR_HIDDEN_FIELDS)
    .sort({ createdAt: -1 });
  return res.status(200).json({ success: true, tickets });
}

async function getTicketById(req, res) {
  const ticket = await Ticket.findById(req.params.id).populate("assignedTo", "name");
  if (!ticket) return res.status(404).json({ success: false, message: "ticket not found" });

  if (req.user.role !== "admin") {
    // Data isolation check: cannot view another author's ticket
    if (ticket.authorId !== req.user.authorId) {
      return res.status(403).json({ success: false, message: "not your ticket" });
    }
    const safe = ticket.toObject();
    delete safe.internalNotes;
    delete safe.aiDraft;
    return res.status(200).json({ success: true, ticket: safe });
  }

  return res.status(200).json({ success: true, ticket });
}
```

F. AI Controller & Draft Caching: server/controllers/aiController.js

// aiController.js — AI Draft Caching & Token Optimization

// aiController.js — AI Draft Caching & Token Optimization

```
async function getDraft(req, res) {
  try {
    const ticket = await Ticket.findById(req.params.id);
    if (!ticket) return res.status(404).json({ success: false, message: "ticket not found" });

    // CACHE CHECK: If draft exists and ?regenerate!=true, return cached draft ($0 cost!)
    if (ticket.aiDraft && req.query.regenerate !== "true") {
      return res.status(200).json({ success: true, draft: ticket.aiDraft });
    }

    // Context assembly: pull ONLY the books relevant to this author
    let books = ticket.bookId
      ? await Book.find({ book_id: ticket.bookId })
      : await Book.find({ authorId: ticket.authorId });

    const author = await User.findOne({ authorId: ticket.authorId });
    const draft = await draftReply(ticket, books, author ? author.name : "there");

    // Persist draft to MongoDB
    ticket.aiDraft = draft;
    await ticket.save();

    return res.status(200).json({ success: true, draft });
  } catch (error) {
    console.error("getDraft error:", error.message);
    // Graceful degradation: allow manual reply if AI fails
    return res.status(503).json({
      success: false,
      message: "AI is not available. Please write the reply manually."
    });
  }
}
```

G. AI Engine & Prompt Engineering: server/AI/

// server/AI/knowledgeBase.js — Sliced Policy Architecture

```
// Sent only when query matches category (saves ~70% prompt tokens)
const policies = {
  "Royalty & Payments": "Royalty: 80% of net profit goes to author. Net profit = MRP - printing - platform commission - shipping. Paid quarterly within 45 days. Minimum payout Rs 1000.",
  "ISBN & Metadata Issues": "ISBN: unique ISBN from BookLeaf imprint. Errors are high priority; promise 48-hour resolution. Metadata updates take 3-5 business days.",
  "Printing & Quality": "Printing: turnaround 5-7 business days. Defective copies get free reprint after verification (request photos from author).",
  "Distribution & Availability": "Distribution: Amazon India, Flipkart, US, UK, BookLeaf Store. Takes 7-10 days to go live. Unavailable status indicates stock sync issue (takes 24-48h).",
  "Book Status & Production Updates": "Stages: Manuscript -> Editing -> Cover Design -> Typesetting -> Proofreading -> ISBN -> Printing -> Distribution -> Published & Live. Be transparent about delays.",
  "General Inquiry": "General: metadata updates can be sent through dashboard or email and take 3-5 business days."
};

const tone = "Tone: empathetic, professional. Acknowledge concern first. Use real numbers/dates. Admit fault if BookLeaf's error. Provide clear timelines and end with next steps.";
```

// server/AI/aiService.js — Strict Parsing & Validation Guardrails

// server/AI/aiService.js — Strict Parsing & Validation Guardrails

```
async function classifyTicket(subject, description) {
  const prompt = `You sort support tickets for BookLeaf Publishing.
Reply with JSON only, like {"category": "...", "priority": "..."}
Categories: ${CATEGORIES.join(", ")}
Priorities: ${PRIORITIES.join(", ")}
Priority rules:
- Critical: money overdue for many months, wrong or duplicate ISBN
- High: royalty looks wrong, book unavailable, defective printed copies
- Medium: production delays, general royalty questions
- Low: simple questions like updating an author bio`;

  const answer = await askAI([
    { role: "system", content: prompt },
    { role: "user", content: `Subject: ${subject}\nDescription: ${description}` }
  ], 60); // 60 token hard cap

  // 1. Strip Markdown fences if present
  const cleaned = answer.replace(/```json|```/g, "").trim();
  const result = JSON.parse(cleaned);

  // 2. Strict Whitelist Safety Check
  if (!CATEGORIES.includes(result.category) || !PRIORITIES.includes(result.priority)) {
    throw new Error("AI gave an invalid category or priority");
  }
  return result;
}
```

3. Comprehensive Technical Interview Q&A

Q1: Why did you use HTTP-only cookies instead of sending Bearer tokens in Authorization headers?

Answer: Bearer tokens stored in browser localStorage or sessionStorage are vulnerable to Cross-Site Scripting (XSS). Any third-party npm package or malicious script injected into the client can execute localStorage.getItem('token') and exfiltrate the credentials. With HTTP-only cookies, the browser transmits the cookie automatically, but client-side JavaScript has zero access to document.cookie, neutralizing token theft.

Q2: What is graceful degradation, and how did you implement it in the AI integration?

Answer: Graceful degradation ensures that when an external dependency (like OpenRouter) fails, the core system continues operating without crashing or blocking users. We implemented this in two key areas: (1) Ticket Creation: AI classification is executed inside a background Promise (.then().catch()). If OpenRouter fails, the catch block logs the error, and the ticket is safely created with default status ('General Inquiry' / 'Medium'). (2) Draft Replies: If the AI API fails, the endpoint returns an HTTP 503 error with an explicit message prompting the support representative to write the reply manually.

Q3: How did you optimize token usage and control operational costs when calling LLMs?

Answer: We engineered three specific cost-saving mechanisms: (1) Sliced Knowledge Base: Instead of dumping the entire company policy manual into every prompt, we inject only the policy slice matching the ticket category, saving ~70% prompt tokens. (2) Strict Max Token Limits: Classification is capped at 60 tokens and response drafting at 400 tokens. (3) Draft Caching: Drafts are saved in MongoDB (ticket.aiDraft). Revisiting a ticket reuses the existing draft at $0 cost; a new API call is only made if the admin explicitly clicks 'Regenerate' (?regenerate=true).

Q4: How does your priority sorting algorithm prioritize tickets in the Admin Queue?

Answer: MongoDB's default string sorting orders values alphabetically ('Critical', 'High', 'Low', 'Medium'), which incorrectly places 'Low' before 'Medium'. In ticketController.js, we defined an explicit ranking dictionary: PRIORITY_RANK = { Critical: 0, High: 1, Medium: 2, Low: 3 }. We then sort with a compound comparator: PRIORITY_RANK[a.priority] - PRIORITY_RANK[b.priority] || a.createdAt - b.createdAt. This ensures all Critical tickets float to the top, and ties are broken by oldest createdAt timestamp.

Q5: How did you prevent authors from seeing internal admin notes or other authors' tickets?

Answer: We implemented defense-in-depth at both the route and query levels. First, authFn middleware validates the JWT and decorates the request with req.user.authorId. When an author fetches their tickets via getMyTickets, we use Mongoose projection .select('-internalNotes -aiDraft') to strip sensitive fields. In getTicketById, we check if (ticket.authorId !== req.user.authorId) and return a 403 Forbidden. Even if an author guesses an ObjectId, they cannot view the data.

Q6: What is BookLeaf's royalty calculation formula, and how did you verify the AI understands it?

Answer: BookLeaf's net profit formula is: Net Profit = MRP minus printing cost, platform commission (Amazon/Flipkart), and shipping charges. The author receives 80% of net profit, and BookLeaf receives 20%. Our prompt explicitly grounds the LLM in this formula. When an author asks why their payout seems lower than total retail copies sold, the AI transparently explains the deductions without being defensive.

Q7: How did you handle the edge cases present in bookleaf_sample_data.json?

Answer: The dataset contained books in production (e.g. 'Midnight in Mysore' and 'Raising Roots') where publication_date and mrp are null, and sales/royalties are 0. In our Mongoose book schema, these fields are optional numbers/strings. On both the backend and frontend, we check publication_date before displaying calculations, preventing NaN errors and presenting an 'In Production' status badge.

Q8: Why did you mark the password field with select: false in the User schema?

Answer: By default, Mongoose returns all schema fields on queries like User.findOne(). If an endpoint serializes user data or logs it, password hashes could leak. Adding select: false completely excludes the password from query results unless explicitly overridden using .select('+password') during login authentication.

Q9: What happens if the LLM classification hallucinates an invalid category or wraps JSON in markdown?

Answer: In classifyTicket(), we implement two guardrails: (1) A regex .replace(/```json|```/g, '') strips markdown code fences before calling JSON.parse(). (2) We validate the parsed output against our hardcoded enums: if (!CATEGORIES.includes(result.category) || !PRIORITIES.includes(result.priority)), it throws an error and falls back to safe system defaults.

Q10: How would you evolve this architecture into an enterprise-scale production system?

Answer: To handle hundreds of thousands of authors: (1) Background Job Queues: Decouple AI calls and email alerts using BullMQ with Redis workers. (2) Database Indexing: Add compound indexes on { authorId: 1, status: 1 } and { priority: 1, createdAt: 1 }. (3) Caching: Cache book catalog metadata in Redis. (4) Cloud Storage: Connect attachment uploads to AWS S3 with pre-signed URLs. (5) Vector DB (RAG): Store BookLeaf's full policy documents in Pinecone/pgvector for semantic policy search.